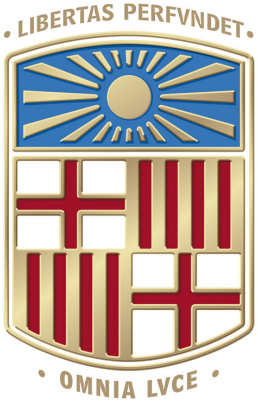

University of Barcelona

Master Data Science: Natural Language Processing

Named Entity Recognition on the GMB Corpus

Benchmarking Classical and Deep Learning Models

Núria Torquet Luna

`https://github.com/ntorqulu/NER_deliverable2.git`

June 20, 2026

Contents

1 Introduction **1**

2 Exploratory Data Analysis **1**

2.1 Dataset overview 1

2.2 Label distribution 1

3 Methodology **2**

3.1 Model 1: CRF 2

3.2 Model 2: Structured Perceptron 3

3.3 Model 3: BiLSTM-CRF 3

3.4 Model 4: BERT 4

4 Experimental Setup **4**

4.1 Training configuration 4

4.2 Evaluation metrics 5

5 Results **6**

5.1 Overall performance 6

5.2 Per-class analysis 6

5.3 TINY TEST results 7

5.4 Analysis of the train-to-test generalization gap 7

6 Conclusions **7**

7 Team Member Contributions **8**

8 Use of AI **8**

1 Introduction

Named Entity Recognition (NER) is a sub-task of information extraction in Natural Language Processing (NLP) that classifies named entities in text into predefined categories such as persons, organizations, locations, time expressions, and more. NER is a foundational component of many NLP applications, including information retrieval, question answering, and relation extraction.

This project benchmarks four machine learning approaches to NER on the *Groningen Meaning Bank* (GMB) corpus in its Kaggle distribution by Walia [2018], spanning from classical feature-based methods to pre-trained transformers:

1. **CRF**: a linear-chain Conditional Random Field with hand-crafted feature templates (classical ML).
2. **Structured Perceptron**: a global linear classifier trained with the averaged perceptron algorithm (classical ML).
3. **BiLSTM-CRF**: a two-layer bidirectional LSTM with a CRF decoding layer (deep learning, trained from scratch).
4. **BERT**: `bert-base-cased` fine-tuned as a token classifier (deep learning, pre-trained transformer).

The dataset uses 17 BIO tags covering eight entity types: geographical entities (`geo`), geopolitical entities (`gpe`), organisations (`org`), persons (`per`), time expressions (`tim`), artefacts (`art`), events (`eve`) and natural phenomena (`nat`).

2 Exploratory Data Analysis

2.1 Dataset overview

The data is split into training and test by sentence identifier, with no sentence appearing in both splits. Table 1 summarizes the basic statistics.

Table 1: Dataset statistics.

Split	Sentences	Tokens	Unique tokens	Mean sent. len
Train	38 366	839 141	31 978	21.9
Test	38 367	837 330	43 954	21.8

An important observation is that the test vocabulary is **37% larger** than the training vocabulary (43 954 vs. 31 978 unique tokens). This gap (approximately 12 000 tokens unseen during training) is the primary issue of the large train-to-test generalization drop observed across all four models, as discussed in Section 5.

2.2 Label distribution

Table 2 shows the per-class token counts. The corpus shows severe class imbalance: the `O` (Outside) tag accounts for approximately 84.7% of all tokens. Among entity tags the most frequent are `B-geo` (3.6%) and `B-tim` (1.9%), while `art`, `eve` and `nat` are extremely rare (<0.1%), making them the hardest classes to learn. The macro-averaged F1 metric penalizes poor performance on these rare classes as heavily as on common ones, which partly explains the low macro F1 values reported in the results.

Table 2: Label distribution in the training set (non-O labels only).

Label	Count	% of all tokens
B-geo	30 113	3.6%
B-tim	16 317	1.9%
B-org	16 127	1.9%
I-per	13 845	1.6%
B-per	13 568	1.6%
I-org	13 526	1.6%
B-gpe	12 739	1.5%
I-geo	5 898	0.7%
I-tim	5 258	0.6%
B-art	314	0.04%
B-eve	244	0.03%
I-art	242	0.03%
I-eve	205	0.02%
I-gpe	161	0.02%
B-nat	157	0.02%
I-nat	44	0.005%

3 Methodology

3.1 Model 1: CRF

A **Conditional Random Field** is a sequence labeling model that, unlike a simple per-token classifier, scores the *entire* tag sequence at once. Given an input sentence $\mathbf{x} = (x_1, \ldots, x_n)$, it assigns a score to every possible tag sequence $\mathbf{y} = (y_1, \ldots, y_n)$ and picks the one with the highest score. The score decomposes over positions and depends on both the current tag and the previous tag, capturing dependencies such as `I-per` can only follow `B-per` or `I-per`, never `B-geo`.

The best tag sequence is found at inference time with the Viterbi algorithm. Training maximizes the conditional log-likelihood of the gold sequences using L-BFGS, an efficient quasi-Newton optimizer.

Custom feature engineering. The strength of a CRF lies entirely in its features. We implement `word2features` in `utils.py`, which extracts the following features at every token position i:

- **Lexical identity.** The lowercase word itself, plus 2, 3 and 4-character suffixes and prefixes. Suffixes like *-istan* or *-ville* are strong location cues; prefixes like *Mac-* or *van* signal surnames.
- **Word shape.** Every character is collapsed to its category: uppercase → `X`, lowercase → `x`, digit → `d`. *Barack* becomes `Xxxxxx`, *NATO* becomes `XXXX`, *9th* becomes `dxx`. Shape features generalise across surface forms.
- **Boolean flags.** Whether the token is all-uppercase, title-case, purely numeric, contains a digit, a hyphen, or a dot; whether it is camelCase; and the ratio of uppercase characters.
- **Context window** $[-2, +2]$**.** Lowercased form, title/upper flags and shape of the two preceding and two following tokens.
- **Bigram context.** Concatenation of the previous word with the current word, and the current word with the next word, capturing two-word patterns such as *New York* or *Prime Minister*.
- **Sentence boundary markers.** `BOS` and `EOS` flags, since sentence-initial capitalization is not a reliable entity signal.

This feature set generates approximately 11 million candidate features. We use L1 and L2 regularization ($c_1 = c_2 = 0.1$), higher than the common default of 0.01, to prune uninformative features and prevent overfitting. `all_possible_states=True` adds a bias weight for every (feature, label) pair even if that combination never appears in training, which is essential for very rare classes such as `B-nat` and `I-nat`.

3.2 Model 2: Structured Perceptron

The **Structured Perceptron** is a global linear classifier that represents the score of a (sentence, tag sequence) pair as a dot product between a weight vector $\mathbf{w}$ and a joint feature vector $\Phi(\mathbf{x}, \mathbf{y})$. The predicted sequence maximizes this score, found with Viterbi decoding.

Training is online and mistake-driven. For each training sentence the model decodes its best guess $\hat{\mathbf{y}}$. If $\hat{\mathbf{y}}$ differs from the original sequence $\mathbf{y}^*$, the weight vector is updated: features active in the original sequence are rewarded (weight $+1$) and features active in the predicted sequence are penalized (weight -1). Training runs for 10 epochs with sentences shuffled at the start of each epoch.

Averaged perceptron. The plain perceptron can oscillate. The **averaged** variant accumulates the sum of every intermediate weight vector across all update steps and divides by the total number of steps at the end, acting as implicit regularization. In our implementation (`train_structured_perceptron` in `utils.py`) averaging is computed with a *lazy cumsum*: rather than iterating over all weights after every sentence (O(|weights|) per step, very slow when running in a laptop with millions of features), we track for each feature the step at which it was last updated and flush its contribution only at update time, reducing the cost to O(updated features).

Custom feature function. `sp_token_features` uses the same feature set as the CRF. Each feature string is concatenated with the current candidate label (e.g. `suffix3=ina::B-geo`), so a single weight vector encodes label-specific lexical preferences. Transition features of the form `prev_label->current_label` play the same role as the CRF transition matrix.

3.3 Model 3: BiLSTM-CRF

The **BiLSTM-CRF** replaces hand-crafted features with automatically learned representations. Our implementation in `train_models.ipynb` is built from scratch using PyTorch and the `pytorch-crf` library, and consists of four stacked components.

1. Embedding layer. Each token is mapped to a 128-dimensional dense vector via a trainable `nn.Embedding` table of size $|\mathcal{V}| \times 128$, where $|\mathcal{V}| = 31{,}980$. Tokens unseen at test time are mapped to the shared `<UNK>` vector, a fundamental limitation for OOV test sets like ours.

2. Bidirectional LSTM. The embedding sequence is fed into a 2-layer bidirectional LSTM. Each layer runs one LSTM left-to-right and one right-to-left; their hidden states are concatenated at every position, giving a 256-dimensional contextualized representation. Dropout ($p = 0.3$) is applied after the embedding and after the LSTM output.

3. Linear emission layer. A single `nn.Linear(256, 17)` layer projects each 256-dim LSTM output to unnormalised emission scores over the 17 BIO tags.

4. CRF decoding layer. The emission scores are passed to a `torchcrf.CRF` layer that learns a 17×17 transition score matrix. During training the CRF computes the negative log-likelihood; at inference `crf.decode` runs Viterbi over emissions plus transitions, guaranteeing valid BIO sequences. Sentences are padded within each batch; a boolean mask excludes padding positions from both the loss and the Viterbi paths.

Training uses AdamW (lr $= 10^{-3}$, weight decay 10^{-4}), gradient clipping (max norm 5.0) and a ReduceLROnPlateau scheduler (patience 3, factor 0.5).

3.4 Model 4: BERT

BERT is a transformer encoder pre-trained on 3.3 billion tokens using masked language modelling. We use `bert-base-cased` (12 layers, 768 hidden dimensions, 110M parameters) and fine-tune it for token classification.

Token classification head. An `AutoModelForTokenClassification` wrapper adds a linear layer $\mathbb{R}^{768} \to \mathbb{R}^{17}$ on top of BERT's final hidden states. The full model is fine-tuned end-to-end.

Sub-word alignment. BERT's WordPiece tokeniser splits words into sub-word pieces (e.g. *Washington* → `[Wash, ##ington]`). Only the **first sub-word token** of each word receives the word's gold label; all continuation pieces receive -100, which is excluded from the cross-entropy loss. At inference, the prediction of the first sub-word token is taken as the word-level prediction.

Why BERT generalises better. The CRF and Structured Perceptron rely on word-identity features that do not fire for OOV tokens. The BiLSTM maps all OOV words to a single random `<UNK>` vector. BERT's sub-word representations, pre-trained on billions of tokens, provide meaningful encodings even for unseen entity surface forms, making it more robust to the 12k OOV tokens in the test set.

Training. AdamW (lr $= 2 \times 10^{-5}$, weight decay 0.01) with 200 linear warmup steps, batch size 16, maximum sequence length 128, for 6 epochs. The best checkpoint is selected by entity-level F1 (seqeval) via `load_best_model_at_end=True`.

4 Experimental Setup

4.1 Training configuration

Each model was trained with a different optimisation strategy due to the nature of its parameters. The CRF and Structured Perceptron are convex models trained to convergence over all data; the deep learning models are trained iteratively with mini-batches and require regularization. Tables 3–6 detail the hyperparameters.

Table 3: CRF training configuration.

Hyperparameter	Value	Rationale
Optimizer	L-BFGS	Quasi-Newton; converges faster than SGD for CRFs
c_1 (L1 penalty)	0.1	Encourages sparsity; zeroes out uninformative features
c_2 (L2 penalty)	0.1	Prevents any single weight from growing too large
Max iterations	200	Upper bound; L-BFGS typically converges before this
`all_possible_transitions`	True	Learns weights for all tag pairs, even unseen ones
`all_possible_states`	True	Per-label bias; essential for rare classes

Table 4: Structured Perceptron training configuration.

Hyperparameter	Value	Rationale
Epochs	10	More passes allow correction of persistent mistakes
Update rule	Averaged perceptron	Averages all weight vectors; acts as regularisation
Averaging	Lazy cumsum	O(updates) instead of O(|weights|) per step
Shuffle	per epoch	Prevents learning a fixed sentence order
Random seed	42	Ensures reproducible shuffle

Table 5: BiLSTM-CRF training configuration.

Hyperparameter	Value	Rationale
Embedding dimension	128	Compact but expressive for a 32k-token vocabulary
Hidden dimension	256	128 per direction; balances capacity and speed
BiLSTM layers	2	Stacking captures higher-order sequential patterns
Dropout	0.3	Applied after embedding and LSTM output
Optimiser	AdamW	Decoupled weight decay; more stable than Adam
Learning rate	10^{-3}	Standard starting point for randomly initialised models
Weight decay	10^{-4}	Light L2 regularisation
LR scheduler	ReduceLROnPlateau	Halves LR when training loss stagnates
Patience / factor	3 / 0.5	Wait 3 epochs; multiply LR by 0.5
Gradient clipping	max norm 5.0	Prevents exploding gradients
Epochs	20	Sufficient for convergence
Batch size	32	Balances memory usage and gradient noise

Table 6: BERT fine-tuning configuration.

Hyperparameter	Value	Rationale
Base model	`bert-base-cased`	Cased model preserves capitalisation
Optimiser	AdamW	Standard for transformer fine-tuning
Learning rate	2×10^{-5}	Small LR avoids destroying pre-trained weights
Weight decay	0.01	Standard L2 regularisation
Warmup steps	200	Linear warmup stabilises early training
Batch size	16	Largest feasible on Colab CPU/single GPU
Epochs	6	Best checkpoint at epoch 3; extra epochs allow detection
Max sequence length	128	Covers 99% of corpus sentences
Checkpoint selection	seqeval F1	Entity-level F1; stricter than token-level
Mixed precision	fp16 (if GPU)	Halves memory usage when GPU is available

4.2 Evaluation metrics

Following the assignment guidelines, all metrics are computed **exclusively on tokens whose ground-truth label is not `O`**. We report:

- **Accuracy**: fraction of non-O tokens correctly tagged.
- **F1 macro**: unweighted mean of per-class F1 scores; penalizes poor performance on rare classes equally to frequent ones.
- **F1 weighted**: class-support-weighted mean of per-class F1 scores.
- **Confusion matrix**: full 16×16 matrix over non-O classes.
- **Tiny-test accuracy**: accuracy on the provided 13-sentence diagnostic set, computed on non-O ground-truth tokens only.

5 Results

5.1 Overall performance

Table 7 summarizes the results on both splits. All metrics are computed on non-O tokens only.

Table 7: Model performance (non-O tokens only).

Model	Split	Accuracy	F1 Macro	F1 Weighted
CRF	Train	0.6079	0.5208	0.6957
CRF	Test	0.4152	0.3182	0.5106
Struct. Perceptron	Train	0.7914	0.6352	0.8038
Struct. Perceptron	Test	0.6344	0.4179	0.6560
BiLSTM-CRF	Train	0.9562	0.9192	0.9605
BiLSTM-CRF	Test	0.4884	0.3703	0.5386
BERT-base-cased	Train	0.9515	0.8828	0.9563
BERT-base-cased	Test	**0.6678**	**0.5281**	**0.6945**

5.2 Per-class analysis

CRF. The CRF reaches moderate training accuracy (0.608) with most of the error concentrated in two cases. First, the model systematically confuses `I-art` with other classes: it assigns `I-art` to 5 263 true `B-geo` tokens and 2 507 true `B-org` tokens on the training set, and 6 250 true `B-geo` tokens on the test set. This indicates that the L1/L2 regularisation forces many ambiguous tokens towards the `I-art` label as a catch-all. Second, rare classes (`B-nat`, `I-nat`, `B-eve`) achieve very low recall (<15% on test), as expected given their scarcity during training.

Structured Perceptron. The SP achieves the strongest classical-model performance on the test set (accuracy 0.634, F1 macro 0.418). Common classes like `B-gpe` (F1 0.744), `I-per` (F1 0.759), `B-tim` (F1 0.789) are handled well. Rare classes remain difficult: `B-eve` (0.032), `I-art` (0.026), `I-eve` (0.040). The averaged perceptron successfully prevents overfitting that the plain perceptron exhibited in previous iterations.

BiLSTM-CRF. The BiLSTM-CRF shows the most large train-to-test gap of all four models (train accuracy 0.956, test 0.488). Without pre-trained embeddings, the model memorizes training-set entity surface forms and fails drastically on the 12k OOV tokens in the test set. The `B-gpe` class drops from train F1 0.976 to test F1 0.380, a direct consequence of OOV: geopolitical adjectives like country names and nationalities are highly lexical and the `<UNK>` vector provides no useful signal. The CRF decoding layer does guarantee valid BIO sequences at inference but cannot compensate for poor scores on unseen tokens.

BERT. BERT achieves the best test performance across all metrics (accuracy 0.668, F1 macro 0.528, F1 weighted 0.695). Common classes are handled well: `B-geo` (F1 0.772), `I-org` (F1 0.664), `I-per` (F1 0.752), `B-tim` (F1 0.762). Even on rare classes BERT outperforms the other models; `B-eve` reaches F1 0.445, the highest among all models. The smaller train-to-test gap (train accuracy 0.952, test 0.668) compared to the BiLSTM-CRF confirms that pre-trained contextual representations are one of the most important factors in generalizing to unseen forms.

5.3 TINY TEST results

Table 8 reports the accuracy on the 13-sentence diagnostic set. All accuracies are computed on non-O ground-truth tokens only.

Table 8: TINY TEST accuracy (non-O tokens only).

Model	Accuracy
CRF	0.2353
Structured Perceptron	0.7059
BiLSTM-CRF	0.8235
BERT-base-cased	**0.9412**

The TINY TEST reveals important differences not captured by the large-corpus metrics. The CRF collapses almost all tokens to `I-art`, a symptom of the regularization artifact described above. The Structured Perceptron correctly identifies most persons and locations but struggles with organizations (*Microsoft* tagged as `B-org` correctly, but *U.S.A* tagged as `B-org` instead of `B-geo`). The BiLSTM-CRF handles capitalized names well but mislabels *Apple* as `B-per` (company vs. common noun ambiguity). BERT is the only model to correctly tag *Apple* as `B-org` in the company context, demonstrating its more robust contextual understanding.

5.4 Analysis of the train-to-test generalization gap

All models show a significant drop from training to test performance (Table 7). The root cause is mostly the vocabulary shift: the test set introduces approximately 12 000 tokens not seen during training. Named entities in news-wire text are highly lexical; a CRF feature like `word.lower=obama` fires only if the word appeared in training. The BiLSTM similarly maps all unseen words to a single `<UNK>` vector.

BERT is the most robust to this shift because its WordPiece sub-word tokenisation and contextual representations provide non-trivial encodings even for entity strings composed of entirely new character sequences. Pre-training on 3.3 billion tokens means that even rare real nouns are likely to appear in related contexts during pre-training, giving BERT a significant advantage over models trained only on the 38k-sentence GMB training set.

The Structured Perceptron generalizes better than the CRF on the test set despite a simpler optimization procedure, which suggests that the online mistake-driven learning with averaging provides a form of data augmentation not present in batch L-BFGS optimization.

6 Conclusions

This project implemented and evaluated four NER models on the GMB corpus, studying classical feature engineering and deep learning. The key findings are:

1. **BERT generalizes best on the test set.** With test accuracy 0.668 and F1 macro 0.528, BERT outperforms all other models. Its pre-trained contextual sub-word representations are the single most important factor in handling the 37% OOV vocabulary shift between training and test.
2. **The Structured Perceptron is the best classical model.** Test accuracy 0.634 and F1 macro 0.418, both higher than the CRF, demonstrate that online averaged perceptron training generalizes better than batch L-BFGS on this task. The lazy cumulative-sum implementation makes training feasible at scale.
3. **The BiLSTM-CRF suffers most from OOV.** Despite achieving the highest training accuracy (0.956), it drops to only 0.488 on the test set, the largest train-to-test gap of any

model. Without pre-trained embeddings, a BiLSTM memorizes training vocabulary rather than learning transferable entity representations. Adding GloVe or fastText initializations would be a future improvement for this model.

4. **Rare classes are universally difficult.** `art`, `eve` and `nat` entities (each <0.1% of training tokens) achieve near-zero F1 on the test set for all models except BERT, which achieves modest but non-zero recall. These classes drive macro F1 values below what weighted F1 or accuracy would suggest.
5. **The train-to-test gap is an dataset characteristic.** Because sentences are split by identifier and cover different news-wire periods, the vocabulary distributions highly diverge. This is not a modeling failure but a data property that any model must try to solve.

Future directions that could improve test performance include: GloVe or fastText-initialised BiLSTM embeddings; using `dslim/bert-base-NER` (already pre-trained on CoNLL-2003 NER) as the base model; adding a CRF head on top of BERT to enforce globally valid BIO sequences; and augmenting training data with external NER corpora such as CoNLL-2003 or WikiANN.

7 Team Member Contributions

This project was carried out individually by **Núria Torquet Luna**. All components: data preprocessing, feature engineering, model implementation, training, evaluation and report writing were completed by the same person.

8 Use of AI

During the development of this project, AI-based tools were used in the following ways:

- **Claude** was used as a coding assistant to help debug environment and library compatibility issues (e.g. `transformers` API changes, `accelerate` version requirements), suggest improvements to the CRF feature set and the Structured Perceptron averaging implementation, and review the structure of the report.
- All model architectures, hyperparameter choices, training loops and evaluation code were written and verified by the author.
- The analysis, interpretation of results and conclusions are the author's own work.

References

Walia, A. (2018). *Annotated Corpus for Named Entity Recognition* [Dataset]. Kaggle. `https://www.kaggle.com/abhinavwalia95/entity-annotated-corpus`